No.
Date

ex8.

Problem 1.

1. algorithm to compute rev(G) in $O(V+E)$

rev(G) has the ~~the~~ same vertices as G, but every edge is flipped.

rev(G):

Create new graph G' with the same vertex set V

for each edge $(u \to v)$ in G:

add edge $(v \to u)$ to G'

return G'

time complexity: O(visit + add)

2. scc(G) is acyclic

Suppose scc(G) contains a cycle between two distinct strong components $C_1 \to C_2 \to \dots \to C_1$.

This means there exist vertices $u \in C_1$ and $v \in C_2$ such that u can reach v and v can reach u.

But if u can reach v and v can reach u, then u and v are mutually reachable, which means they belong to the same strong component — contradicting the assumption that C_1 and C_2 are distinct components.

Therefore scc(G) can't contain a cycle, and is acyclic

No.
Date

3. $sc(rev(G)) = rev(sc(G))$

we need to show that the two graph have the same vertices and edges.

Same vertices.

The vertices of $sc(G)$ are the strong component of G. we need two show that the strong components of $rev(G)$ are ~~the~~ exactly the same sets of vertices.

A set of vertices C is a strong component of G if and only if verry pair of vertices $u, v \in C$ can reach each other in G, Now, u can reach v in G if and only if v can reach u in $rev(G)$. Therefore, u and v are mutually reachable in G if and only if they are mutually reachable in $rev(G)$. this means C is a strong component of G if and only if C is a strong component of $rev(G)$

So, ~~sc(G) and rev~~ $sc(rev(G))$ and $rev(sc(G))$ have the same vertex sets.

Same edges.

Consider any two strong components C_1 and C_2.

There is an edge $C_1 \to C_2$ in $sc(rev(G))$ if and

No.
Date

only if there exists an edge $(u \to v)$ in $rev(G)$ with $u \in C_1$ and $v \in C_2$, which happens if and only if there exists an edge $(v \to u)$ in G with $v \in C_2$ and $u \in C_1$, which means there is an edge $C_2 \to C_1$ in $scc(G)$, which means there is an edge $C_1 \to C_2$ in $rev(G)$.

So the edey sets are identical.

Conclusion

Since $scc(rev(G))$ and $rev(scc(G))$ have the same vertice and the same edges they are equal.

$$scc(rev(G)) = rev(scc(G))$$

4. u reaches v in $G \Rightarrow S(u)$ reaches $S(v)$ in $scc(G)$

direction 1: If u reaches v in G, then $S(u)$ reaches $S(v)$ in $scc(G)$

Suppose u can reach v in G via some path:

$$u = u_0 \to u_1 \to u_2 \to \dots \to u_k = v$$

each consecutive pair $u_i \to u_{i+1}$ is an edge in G, in $scc(G)$ each vertex u_i belongs to some strong component $S(u_i)$. For each edge $u_i \to u_{i+1}$, either.

No.

Date

$S(u_i) = S(u_{i+1})$, same component, no new edge needed, or $S(u_i) \neq S(u_{i+1})$, in which case there is an edge $S(u_i) \to S(u_{i+1})$ in $scc(G)$

Following this chain from $S(u_0) = S(u)$ to $S(u_k) = S(v)$, we get a path from $S(u)$ to $S(v)$ in $scc(G)$.

direction 2: if $S(u)$ reaches $S(v)$ in $scc(G)$, then u reaches v in G.

Suppose $S(u)$ can reach $S(v)$ in $SSC(G)$ via some path of components.

$$S(u) = C_0 \to C_1 \cdots C_m = S(v)$$

each edge $C_i \to C_{i+1}$ in $scc(G)$ exist, because there is at least one edge $(x \to y)$ in G with $x \in C_i$ and $y \in C_{i+1}$

Since $u \in C_0$ and $x \in C_0$ are in the same strong component, u can reach x in G, since $y \in C_1$ and the next representive of C_1 are in the same strong component we can keep chaining across component all the way to some vertex in $C_m = S(v)$

finally, since $v \in S(v)$ and all vertex in $S(v)$ are mutually reachable, we can reach v.

There fore u can reach v in G

No.

Date

Problem 2.

1. G has an Euler tour $\Leftrightarrow$ in-degree(v) = out-degree(v) for v

direction ($\Rightarrow$): in a closed tour using every edge once, every time we enter a vertex we must leave it, so in-degree = out degree for every vertex

direction ($\Leftarrow$): start anywhere, follow unused edges until stuck — since degree are balanced, we always reture to the start, forming a closed walk. If unused edge remain, find a vertex on the walk unused edges, from another closed walk from there, and merge it in. Repeat until all edges are used.

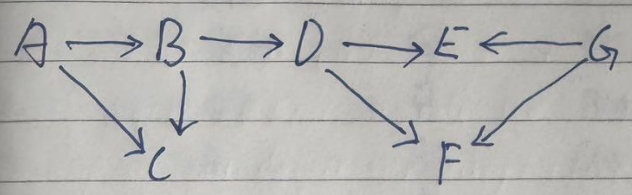
2. algorithm.

1. start at any vertex, follow unused edges until back at start → cycle C.
2. while unused edge exist.
   pick any vertex u in C with unused edge,
   follow unused edges from u to u → cycle C'
   merge C' into C at u
3. Return C.

No.

Date

Problem 3.



Sorting 1. start from A.

In-degree : A=0, B=1, C=2, D=1, E=3, F=2 G=0

Step	queue	Pick	remove edges, update in-degre
1	{A, G}	A	B:0, C:1
2	{G, B}	B	C:0, D:0
3	{G, C, D}	C	E:2
4	{G, D}	D	E:1, F:1
5	{G}	G	E:0, F:0
6	{E, F}	E	
7	{F}	F	

result 1: A→B→C→D→G→E→F

No.

Date

Setting 2: start from G.

step	queue	pick	update
1	{A, G}	G	E:2, F:1
2	{A}	A	B:0, C:1
3	{B}	B	C:0, D:0
4	{C, D}	D	E:1, F:0
5	{C, F}	C	E:0
6	{F, E}	F	
7	{E}	E	

result: G → A → B → D → C → F → E